Security Assessment Report

Exponent Core Simplified

May 15, 2026

Summary

The Sec3 team was engaged to conduct a thorough security analysis of the Exponent Core Simplified.

The artifact of the audit was the source code of the following programs, excluding tests, in a private repository.

The initial audit focused on the following versions and revealed 6 issues or questions.

Task	Type	Commit
exponent_core_simple	Solana	`2d6bbaf` (11/10/2025)

This report provides a detailed description of the findings and their respective resolutions.

Table of Contents

Result Overview

Issue	Impact	Status
EXPONENT_CORE_SIMPLE		
[M-01] Emission rewards accrue incorrectly in emergency mode	Medium	Resolved
[M-02] Missing minimum YT deposit check enables interest griefing	Medium	Acknowledged
[L-01] Redundant `RemoveVaultEmission` action in the `ModifyVaultSetting`	Low	Acknowledged
[L-02] `withdraw_yt` in emergency mode causes irrecoverable interest loss	Low	Acknowledged
[I-01] Missing constraint on `treasury_fee_bps` in `AddEmission` instruction	Info	Resolved
[I-02] Missing validation for `interest_bps_fee` in `InitializeVault` instruction	Info	Resolved

Findings in Detail

EXPONENT_CORE_SIMPLE

[M-01] Emission rewards accrue incorrectly in emergency mode

Identified in commit `2d6bbaf`.

In the current program, user rewards from the `sy` deposit consist of two parts — `sy_interest` and `emissions`. The functions `DepositYt`, `WithdrawYt`, and `StageYield` can update the vault state and invoke `earn_all` to update the user's rewards.

```
/* solana/programs/exponent_core_simple/src/state/yield_token_position.rs */
094 | /// Stage earned SY and rewards
095 | fn earn_all(&mut self, vault: &mut Vault) {
096 |     self.earn_sy_interest(vault);
097 |
098 |     self.earn_emissions(vault);
099 | }
```

For the vault, if the `last_seen_sy_exchange_rate` is lower than the `all_time_high_sy_exchange_rate`, the vault enters emergency mode as the `sy_exchange_rate` is not increased as expected. While in emergency mode, both `DepositYield` and `StageYield` will reject any operations that update the vault or calculate earnings for yield position. However, `WithdrawYt` can still be executed in emergency mode.

```
/* solana/programs/exponent_core_simple/src/state/vault.rs */
119 | /// Emergency mode is if the vault's all-time-high is greater than the last seen exchange rate
120 | pub fn is_in_emergency_mode(&self) -> bool {
121 |     self.all_time_high_sy_exchange_rate > self.last_seen_sy_exchange_rate
122 | }

/* solana/programs/exponent_core_simple/src/instructions/vault/deposit_yt.rs */
190 | // Do not allow deposits if the current sy exchange rate is lower than the all-time high
191 | // This prevents users from increasing their position amount with a lower last_seen_index, which would break
↪  the economics.
192 | require!(
193 |     !vault.is_in_emergency_mode(),
194 |     ExponentCoreError::VaultInEmergencyMode
195 | );

/* solana/programs/exponent_core_simple/src/instructions/vault/stage_yield.rs */
118 | // TODO - consider removing this check, since deeper in the stack we check for this
119 | require!(
120 |     !vault.is_in_emergency_mode(),
121 |     ExponentCoreError::VaultInEmergencyMode
122 | );

/* solana/programs/exponent_core_simple/src/instructions/vault/withdraw_yt.rs */
181 | pub fn handle_withdraw_yt(
```

```
182 |     vault: &mut Vault,
183 |     vault_yield_position: &mut YieldTokenPosition,
184 |     user_yield_position: &mut YieldTokenPosition,
185 |     sy_state: &SyState,
186 |     now: u32,
187 |     amount: u64,
188 | ) {
189 |     update_vault_yield(vault, vault_yield_position, now, sy_state);
190 |
191 |     // Note that withdraw YT only can occur if the vault is active
192 |     yield_position_earn(vault, user_yield_position);
```

For the two functions inside `earn_all`:

- `earn_sy_interest`. When the vault is in `emergency_mode`, the function immediately returns without calculating interest and without updating `interest.last_seen_index` in the yield position.

```
/* solana/programs/exponent_core_simple/src/state/yield_token_position.rs */
067 | fn earn_sy_interest(&mut self, vault: &mut Vault) {
068 |     // If vault is in emergency mode, do nothing
069 |     if vault.is_in_emergency_mode() {
070 |         return;
071 |     }
```

- `earn_emissions`. This operation continues to execute normally.

```
/* solana/programs/exponent_core_simple/src/state/yield_token_position.rs */
101 | fn total_sy_balance(&self, vault: &Vault) -> u64 {
102 |     self.interest.staged + py_to_sy(vault.final_sy_exchange_rate, self.yt_balance)
103 | }
104 |
105 | fn earn_emissions(&mut self, vault: &Vault) {
106 |     // TODO - consider negative rates
107 |     let sy_balance = self.total_sy_balance(vault);
108 |
109 |     for (index, emission) in vault.emissions.iter().enumerate() {
110 |         let e = &mut self.emissions[index];
111 |         let earned_emission =
112 |             calc_share_value(e.last_seen_index, emission.final_index, sy_balance);
113 |
114 |         e.inc_staged(earned_emission);
115 |         e.last_seen_index = emission.final_index;
116 |     }
117 | }
```

As the `WithdrawYt` instruction does not prevent `earn_all` in emergency mode, emissions can still be updated, and rewards can be computed based on an inflated `sy_balance`. Specifically:

```
sy_balance = interest.staged + (yt_balance / vault.final_sy_exchange_rate)
earned_emission = (emission.final_index - e.last_seen_index) * sy_balance
```

For `earn_emissions`, the valuation of `sy_balance` becomes inflated when the `final_sy_exchange_rate` de-

creases, allowing users to unexpectedly earn more emission rewards. Subsequently, attackers can directly withdraw emission rewards recorded in `staged` via `collect_emissions`.

This issue will affect the vault when it is active. When the vault enters the expired state, the `emission.final_index` will no longer be updated, and all newly generated rewards will belong to the vault.

It is recommended to add a check for `vault.is_in_emergency_mode()` within `earn_emissions` as well, returning immediately to prevent users from accumulating rewards while in emergency mode.

Resolution

This issue has been fixed by `a361106`.

EXPONENT_CORE_SIMPLE

[M-02] Missing minimum YT deposit check enables interest griefing

Identified in commit `2d6bbaf`.

1. `DepositYt`

In the `DepositYt` instruction, users are allowed to deposit `YT` tokens into any `YieldTokenPosition`. During this process, the protocol accrues `SY` interest to the target position using the formula: `floor(yt_balance * (1 / sy_exchange_rate_last_seen - 1 / sy_exchange_rate_cur))`.

```
/* solana/programs/exponent_core_simple/src/instructions/vault/deposit_yt.rs */
102 | pub fn handler(ctx: Context<DepositYt>, amount: u64) -> Result<DepositYtEventV2> {
103 |     let current_unix_timestamp = now();
104 |
105 |     let sy_state = do_get_sy_state(
106 |         &ctx.accounts.address_lookup_table.to_account_info(),
107 |         &ctx.accounts.vault.cpi_accounts,
108 |         ctx.remaining_accounts,
109 |         ctx.accounts.sy_program.key(),
110 |     )?;
111 |
112 |     handle_deposit_yt(
113 |         &mut ctx.accounts.vault,
114 |         &mut ctx.accounts.yield_position,
115 |         &mut ctx.accounts.user_yield_position,
116 |         &sy_state,
117 |         current_unix_timestamp,
118 |         amount,
119 |     )?;

/* solana/programs/exponent_core_simple/src/state/yield_token_position.rs */
201 | /// Calculate the amount of SY earned from an appreciation in the exchange rate
202 | fn calc_earned_sy(
203 |     yt_balance: u64,
204 |     sy_exchange_rate_last_seen: Number,
205 |     sy_exchange_rate_cur: Number,
206 | ) -> u64 {
207 |     // If the last seen exchange rate is equal (or higher?!) than the final rate, we're done
208 |     // or if there is 0 yt balance
209 |     // or if the last seen exchange rate is 0 (which should not happen if YT balance is not 0 anyway)
210 |     if sy_exchange_rate_last_seen >= sy_exchange_rate_cur
211 |         || yt_balance == 0
212 |         || sy_exchange_rate_last_seen == Number::ZERO
213 |     {
214 |         return 0;
215 |     }
216 |
217 |     let delta = Number::ONE / sy_exchange_rate_last_seen - Number::ONE / sy_exchange_rate_cur;
218 |
219 |     // number of "SY shares" earned from the pool
220 |     let sy_earned = delta * yt_balance.into();
221 |
222 |     sy_earned.floor_u64()
```

```
223 | }
```

Here, `yt_balance` is the deposited `YT` balance of the target position, and the difference between `sy_exchange_rate_cur` and `sy_exchange_rate_last_seen` reflects the yield growth since the last update.

However, because there is no minimum deposit check, a malicious user could repeatedly deposit zero-value `YT` tokens into a victim's `YieldTokenPosition`. Since the rate change between updates may be small, this can cause the interest calculation to round down to zero, effectively preventing the victim from accruing yield. Over time, this creates a griefing vector that erodes user rewards.

It is recommended to add a minimum `YT` deposit threshold to prevent zero-value deposits and mitigate interest griefing attacks.

2. `StageYield`

The `stage_yield` instruction also accrues `SY` interest. Since this instruction is permissionless, it shares the same vulnerability.

```
/* solana/programs/exponent_core/src/instructions/vault/stage_yield.rs */
009 | pub struct StageYield<'info> {
          // @audit: permissionless
010 |     #[account(mut)]
011 |     pub payer: Signer<'info>,
044 | }

046 | pub fn handler(ctx: Context<StageYield>) -> Result<StageYieldEventV2> {
047 |     let sy_state = do_get_sy_state(
048 |         &ctx.accounts.address_lookup_table.to_account_info(),
049 |         &ctx.accounts.vault.cpi_accounts,
050 |         ctx.remaining_accounts,
051 |         ctx.accounts.sy_program.key(),
052 |     )?;
053 |
054 |     handle_stage_yt_yield(
055 |         &mut ctx.accounts.vault,
056 |         &mut ctx.accounts.yield_position,
057 |         &mut ctx.accounts.user_yield_position,
058 |         &sy_state,
059 |         now(),
060 |     )?;
077 |     Ok(event)
078 | }

107 | pub fn handle_stage_yt_yield(
113 | ) -> Result<()> {
114 |     // update vault indexees from SY state
115 |     // and stage any yield to the vault's robot account
116 |     update_vault_yield(vault, vault_yield_position, now, sy_state);
124 |     yield_position_earn(vault, user_yield_position);
130 | }
```

Resolution

The team acknowledged this finding.

EXPONENT_CORE_SIMPLE

[L-01] Redundant `RemoveVaultEmission` action in the `ModifyVaultSetting`

Identified in commit `2d6bbaf`.

In the `ModifyVaultSetting` instruction, the admin can remove a vault emission via the `RemoveVaultEmission` action. This operation removes the element at the target index in the emissions vector and shifts all subsequent elements to the left.

```
/* solana/programs/exponent_core_simple/src/instructions/vault/admin/modify_vault_setting.rs */
195 | AdminAction::RemoveVaultEmission(emission_index) => {
196 |     ctx.accounts
197 |         .admin_state
198 |         .principles
199 |         .exponent_core
200 |         .is_admin(ctx.accounts.signer.key)?;
201 |
202 |     vault.emissions.remove(emission_index as usize);
203 | }
```

During the `update_from_sy_state` process, the vault synchronizes its emission data using the `sy_state` fetched from the SY program. It iterates over all items in `sy_state.emission_indexes` and updates `vault.emissions` in order based on the index.

```
/* solana/programs/exponent_core_simple/src/state/vault.rs */
356 | for (index, x) in sy_state.emission_indexes.iter().enumerate() {
357 |     self.emissions[index].last_seen_index = *x;
358 |
359 |     // if the vault is active, then update the final index
360 |     if self.is_active(now) {
361 |         self.emissions[index].final_index = *x;
362 |     }
363 | }
```

However, in the current `generic_standard` implementation (the SY program), there is only an `add_emission` instruction and no `remove_emission` instruction. This means emissions in the `sy_state` are never removed. If a core program admin mistakenly executes `RemoveVaultEmission` inside `ModifyVaultSetting`, it will shift elements inside `vault.emissions`, breaking the positional alignment between `vault.emissions` and `sy_state.emission_indexes`. As a result, `update_from_sy_state` can no longer function correctly.

It is recommended to remove the unnecessary `RemoveVaultEmission` action from `ModifyVaultSetting`.

Resolution

The team acknowledged this finding.

EXPONENT_CORE_SIMPLE

[L-02] `withdraw_yt` in emergency mode causes irrecoverable interest loss

Identified in commit `2d6bbaf`.

The `withdraw_yt` instruction allows users to withdraw their YT. In terms of accounting, it moves the YT from the user's position to the vault's position. This action does not transfer actual ownership of the YT to the vault; rather, it transfers the rights to future yield. Until this specific portion of YT is deposited back into the program, all associated yield accrues to the vault.

```
/* solana/programs/exponent_core/src/instructions/vault/withdraw_yt.rs */
181 | pub fn handle_withdraw_yt(
182 |     vault: &mut Vault,
183 |     vault_yield_position: &mut YieldTokenPosition,
184 |     user_yield_position: &mut YieldTokenPosition,
185 |     sy_state: &SyState,
186 |     now: u32,
187 |     amount: u64,
188 | ) {
189 |     update_vault_yield(vault, vault_yield_position, now, sy_state);
190 |
191 |     // Note that withdraw YT only can occur if the vault is active
192 |     yield_position_earn(vault, user_yield_position);
193 |
194 |     user_yield_position.dec_yt_balance(amount);
195 |     vault_yield_position.inc_yt_balance(amount);
196 |
197 |     // After increasing uncollected_sy, set_sy_for_pt
198 |     vault.set_sy_for_pt();
199 | }
```

Under normal circumstances, we expect the `interest.last_seen_index` for both positions to be updated to the latest value before transferring yield rights. This ensures all accrued interest is correctly settled up to the current rate.

However, since this instruction functions during emergency mode, the execution chain `yield_position_earn` - `earn_all_with_tracking` - `earn_all` - `earn_sy_interest` skips interest settlement and index updates. This results in a discrepancy between the `interest.last_seen_index` of the `user_yield_position` and the `vault_yield_position`, leading to chaotic interest calculations later.

```
/* solana/programs/exponent_core/src/state/yield_token_position.rs */
067 | fn earn_sy_interest(&mut self, vault: &mut Vault) {
068 |     // If vault is in emergency mode, do nothing
069 |     if vault.is_in_emergency_mode() {
070 |         return;
071 |     }
```

Specifically, the `vault_yield_position` attempts to synchronize its `interest.last_seen_index` whenever the vault exchange rate updates. Since the index only increases, the vault's index represents the highest historical exchange rate encountered. The `user_yield_position`, however, is not updated as frequently and likely holds an older, lower value. Consequently, when the indices are not synchronized (as happens in emergency mode), YT is transferred from a position with a lower `last_seen_index` to one with a higher index. Even after emergency mode ends, the interest accrued between these two indices is skipped and lost.

Accrued interest is tracked internally via `interest.staged`, which will be lower than it should be due to this issue. However, the amount of SY redeemable by PT is calculated using the initial base asset value divided by the current SY exchange rate (as PT are zero-coupon bonds), rather than by subtracting the vault's interest from the total balance. As a result, the "lost" interest remains in the vault but becomes unclaimable by any party.

```
/* solana/programs/exponent_core/src/state/vault.rs */
570 | fn sy_backing_for_pt(sy_exchange_rate: Number, pt_supply: u64, sy_in_escrow: u64) -> u64 {
571 |     if sy_exchange_rate == Number::ZERO {
572 |         return 0;
573 |     }
574 |     let sy_normal = Number::from_natural_u64(pt_supply) / sy_exchange_rate;
575 |     let sy_normal = sy_normal.floor_u64();
576 |     let sy_fallback = sy_in_escrow;
577 |
578 |     // the amount of SY set aside for PT holders is the minimum of the two
579 |     let sy = sy_normal.min(sy_fallback);
580 |
581 |     sy
582 | }
```

This issue is difficult to fix gracefully. The most direct solution is to prohibit calling `withdraw_yt` while the vault is in emergency mode.

Resolution

The team acknowledged this finding.

EXPONENT_CORE_SIMPLE

[I-01] Missing constraint on `treasury_fee_bps` in `AddEmission` instruction

Identified in commit `2d6bbaf`.

The `AddEmission` instruction does not enforce a `treasury_fee_bps <= 10000` constraint.

```
/* solana/programs/exponent_core_simple/src/instructions/vault/admin/add_emission.rs */
065 | #[access_control(ctx.accounts.validate())]
066 | pub fn handler<'info>(
067 |     ctx: Context<'_, '_, '_, 'info, AddEmission<'info>>,
068 |     cpi_accounts: CpiAccounts,
069 |     treasury_fee_bps: u16,
070 | ) -> Result<()> {
083 |     ctx.accounts.vault.add_emission(
084 |         ctx.accounts.robot_token_account.key(),
085 |         &sy_state,
086 |         ctx.accounts.treasury_token_account.key(),
087 |         treasury_fee_bps,
088 |     );
```

When `treasury_fee_bps` exceeds 10000 and is stored in `vault.emissions[index as usize].fee_bps`, it may trigger transaction failures during `collect_emission`. This occurs due to an arithmetic validation at line 182 in `collect_emission.rs`.

```
/* solana/programs/exponent_core_simple/src/instructions/vault/collect_emission.rs */
088 | #[access_control(ctx.accounts.validate())]
089 | pub fn handler(
090 |     ctx: Context<CollectEmission>,
091 |     index: u16,
092 |     amount: Amount,
093 | ) -> Result<CollectEmissionEventV2> {
110 |     let (user_amount, treasury_amount) = handle_collect_emission(
111 |         amount_to_send,
112 |         ctx.accounts.vault.emissions[index as usize].fee_bps,
113 |     )?;

/* solana/programs/exponent_core_simple/src/instructions/vault/collect_emission.rs */
172 | fn handle_collect_emission(amount_to_send: u64, fee_bps: u16) -> Result<(u64, u64)> {
173 |     let fee_bps_u64 = fee_bps as u64;
174 |
175 |     let treasury_amount = (amount_to_send * fee_bps_u64 + 9999) / 10000;
176 |
177 |     // Calculate the user amount rounded down
178 |     let user_amount = amount_to_send.saturating_sub(treasury_amount);
179 |
180 |     // Assert that the sum of user_amount and treasury_amount equals amount_to_send
181 |     assert!(
182 |         user_amount + treasury_amount == amount_to_send,
183 |         "The sum of user_amount and treasury_amount should be equal to amount_to_send"
184 |     );
185 |
186 |     Ok((user_amount, treasury_amount))
```

```
187 | }
```

We suggest adding a bounds check in `AddEmission`, similar to the validation used in `ModifyVaultSetting`:

```
assert!(
    treasury_fee_bps <= 10000,
    "Fee BPS must be less than or equal to 10000"
);
```

Resolution

This issue has been fixed by `19faed9`.

EXPONENT_CORE_SIMPLE

[I-02] Missing validation for `interest_bps_fee` in `InitializeVault` instruction

Identified in commit `2d6bbaf`.

The `InitializeVault` instruction lacks validation to ensure `interest_bps_fee` does not exceed 10,000.

```
/* solana/programs/exponent_core_simple/src/instructions/vault/admin/initialize_vault.rs */
138 | impl InitializeVault<'_> {
139 |     fn set_vault(
140 |         &mut self,
141 |         start_timestamp: u32,
142 |         duration: u32,
143 |         cpi_accounts: CpiAccounts,
144 |         address_lookup_table: Pubkey,
145 |         vault_signer_bump: u8,
146 |         interest_bps_fee: u16,
147 |         min_op_size_strip: u64,
148 |         min_op_size_merge: u64,
149 |     ) {
175 |         vault.interest_bps_fee = interest_bps_fee;
```

Excessively high `interest_bps_fee` values may trigger a panic in `calc_collect_interest` due to the `checked_sub` operation.

```
/* solana/programs/exponent_core_simple/src/instructions/vault/collect_interest.rs */
196 | fn calc_collect_interest(amount_sy: u64, interest_bps_fee: u16) -> (u64, u64) {
197 |     let fee_sy = (amount_sy as u128 * interest_bps_fee as u128 + 9999) / 10000;
198 |     let fee_sy = fee_sy as u64;
199 |
200 |     let user_sy = amount_sy.checked_sub(fee_sy).unwrap();
201 |
202 |     (user_sy, fee_sy)
203 | }
```

We recommend implementing a bounds check in `InitializeVault`, mirroring the validation in `ModifyVaultSetting`:

```
assert!(
    interest_bps_fee <= 10000,
    "Fee BPS must be less than or equal to 10000"
);
```

Resolution

This issue has been fixed by `e5a49fd`.

Appendix: Methodology and Scope of Work

Assisted by the Sec3 Scanner developed in-house, the manual audit particularly focused on the following work items:

- Check common security issues.
- Check program logic implementation against available design specifications.
- Check poor coding practices and unsafe behavior.
- The soundness of the economics design and algorithm is out of scope of this work

DISCLAIMER

The instance report ("Report") was prepared pursuant to an agreement between Coderrect Inc. d/b/a Sec3 (the "Company") and Exponent Labs Ltd. (the "Client''). This Report solely includes the results of a technical assessment of a specific build and/or version of the Client's code specified in the Report ("Assessed Code") by the Company. The sole purpose of the Report is to provide the Client with the results of the technical assessment of the Assessed Code. The Report does not apply to any other version and/or build of the Assessed Code. Regardless of the contents of the Report, the Report does not (and should not be interpreted to) provide any warranty, representation or covenant that the Assessed Code: (i) is error and/or bug free, (ii) has no security vulnerabilities, and/or (iii) does not infringe any third-party rights. Moreover, the Report is not, and should not be considered, an endorsement by the Company of the Assessed Code and/or of the Client. Finally, the Report should not be considered investment advice or a recommendation to invest in the Assessed Code and/or the Client.

This Report is considered null and void if the Report (or any portion thereof) is altered in any manner.

ABOUT

The Sec3 audit team comprises a group of computer science professors, researchers, and industry veterans with extensive experience in smart contract security, program analysis, testing, and formal verification. We are also building automated security tools that incorporate static analysis, penetration testing, and formal verification.

At Sec3, we identify and eliminate security vulnerabilities through the most rigorous process and aided by the most advanced analysis tools.

For more information, check out our website and follow us on twitter.